

```

1 %=====
2 % Deterministic Cake-Eating Problem: Value Function Iteration
3 %=====
4
5 clear
6 clc
7
8 %LaTeX Interpreter if ==1 (requires LaTeX to be installed if compiled on Octave)
9 latex_interpreter=0;
10
11 %-----
12 % CRRA Utility Function
13 %-----
14 gamma=1; %gamma>0
15 if gamma==1
16     u = @(c)log(c);
17 else
18     u = @(c) (c.^(1-gamma)-1)/(1-gamma);
19 end
20
21 %-----
22 % Uniform Grid on Cake Size
23 %-----
24 dimW=2000; %number of nodes
25 gridW=linspace(0.005,1,dimW); %uniform grid over cake size
26 % It starts with 0.005 in case u(0) is not well-defined.
27
28 %-----
29 % Defining the Auxiliary Matrix of Consumption
30 % for Each Combination of Today and Tomorrow's States
31 %-----
32 % Consumption can be recovered from current and next period states.
33 % Creating the (dimW,dimW)-matrix such that an (i,j) element is
34 % the amount of consumption associated with
35 % w=gridW(i) (current state is the i-th node) and
36 % w_prime=gridW(j) (next period's state is the j-th node).
37 % Note that a choice variable is the next-period state.
38 consumption = gridW'*ones(1,dimW)-ones(dimW,1)*gridW;
39 consumption(consumption<=0)=0.1^20;
40 % gridW'*ones(1,dimW) yields the (dimW,dimW) matrix ( gridW' ... gridW' )
41 % ones(dimW,1)*gridW yields the (dimW,dimW) matrix ( gridW; ... gridW )
42 % Replace non-positive (infeasible) consumption with
43 % something sufficiently close to zero.
44
45 %-----
46 % Setting Parametric Values for the Value Function Iteration
47 %-----
48 beta=0.85; %Discount Factor
49 max_iter=30; %The Maximum Number of Iterations
50 V=zeros(max_iter+1,dimW); %Value functions
51
52 %-----
53 % Value function Iteration
54 %-----
55 tic %start stopwatch timer (optional)
56 for i=1:max_iter
57     [V(i+1,:),index]=max( (u(consumption) + beta*ones(dimW,1)*V(i,:))' );
58 end %end of for

```

```

59 comp_time=toc; %end stopwatch timer and substitute time into comp_time (optional)
60 fprintf(' Time: %f seconds.\n', comp_time);
61
62 %-----
63 % Plotting Value functions
64 %-----
65 exact_V= ((log(1-beta)/(1-beta))+ (beta/((1-beta)^2))*log(beta))*ones(1,dimW)+ (log(gridW)./(1-
    beta));
66 figure()
67 hold on
68 grid on
69 plot(gridW, V,'LineWidth',1);
70 plot(gridW, exact_V,'k--','LineWidth',1);
71 hold off
72 if latex_interpreter==1
73     title('Value Function Iteration','Interpreter','latex','FontSize',14);
74     xlabel('Size of Cake','Interpreter','latex','FontSize',12);
75     ylabel('Value','Interpreter','latex','FontSize',12);
76 else
77     title('Value Function Iteration','FontSize',14);
78     xlabel('Size of Cake','FontSize',12);
79     ylabel('Value','FontSize',12);
80 end
81

```